STM32CubeIDE

One Day Intensive Course

What the IDE is hiding — and how to own every layer

Week 2 · Firmware 60-Day Challenge
Ethan · July 2026

Topics covered in this course

- Block 1 — The build pipeline: ARM GCC, flags, and your first terminal build
- Block 2 — The linker script and startup file: how your program begins before main()
- Block 3 — Binutils autopsy: reading your own ELF with objdump, nm, and size
- Block 4 — Debug plumbing: ST-Link, SWD, OpenOCD, and raw GDB
- Block 5 — Makefile from scratch, then mapping it all to PlatformIO

Resources referenced throughout

- Memfault Interrupt blog — 'Zero to main()' series (interrupt.memfault.com)
- bare-metal-programming-guide by Sergey Lyubka (github.com/cpq/bare-metal-programming-guide)
- STM32F446 Reference Manual RM0390 (st.com)
- Phil's Lab YouTube channel (@PhilsLab)
- Israel Gbati — 'Embedded Systems Bare-Metal Programming Ground Up (STM32)' (Udemy)

Your Prompt

> *"Eclipse-based IDE wrapping the ARM GCC toolchain, OpenOCD, and ST-Link drivers. Understanding what CubeIDE is hiding makes the transition to VS Code + PlatformIO trivial later. This is something you told me to learn for week 2. You are now a world class teacher. Give me a one day intensive course on this. Include any resources if they would help explain it better."*

Block 1 (09:00–10:30) — The Build Pipeline

CubeIDE is a shell around five command-line tools that have existed for decades. Understanding each one means the IDE's build output stops being noise and starts being a readable log.

The five tools

- **arm-none-eabi-gcc** — the cross-compiler. Turns your C into ARM Thumb machine code. The 'none-eabi' means it targets bare metal (no OS, embedded ABI).
- **arm-none-eabi-as** — assembler. Usually driven transparently by gcc via the -x assembler flag.
- **arm-none-eabi-ld** — linker. Combines all .o files and assigns real memory addresses using the linker script.
- **arm-none-eabi-objcopy** — converts the ELF output into a raw .bin or Intel .hex image suitable for flashing.
- **arm-none-eabi-gdb** — the debugger. GDB itself; CubeIDE's debug perspective is a GUI sitting on top of it.

The build pipeline in order

Source files (.c, .s) → arm-none-eabi-gcc (compile + assemble) → .o object files → arm-none-eabi-ld (link, using the .ld script) → firmware.elf → arm-none-eabi-objcopy → .bin / .hex (raw flash image).

The .elf file is for debugging — it carries your machine code plus debug symbols that map instructions back to C lines. GDB reads the .elf. The .bin is exactly the bytes written into flash, nothing more.

Key compiler flags to understand

- **-mcpu=cortex-m4 -mthumb** — target the M4 core, generate Thumb-2 instructions.
- **-mfloat-abi=hard -mfpu=fpv4-sp-d16** — use the hardware FPU (M4 only); wrong ABI = crash.
- **-ffunction-sections -fdata-sections** — put each function/variable in its own ELF section so the linker can dead-strip unused code via --gc-sections.
- **-Og -g3** — optimize for debugging, maximum debug info.
- **-nostdlib / --specs=nosys.specs** — don't link the standard C library (bare metal has no OS syscalls).

Hands-on (30 min)

- Find the toolchain CubeIDE hides — on Windows under the CubeIDE install in a plugins/…gnu-tools-for-stm32… folder. Locate arm-none-eabi-gcc and run --version from a terminal.
- Build any project in CubeIDE. Read the Console tab slowly — every line is a real command. Copy one full compile command into a notes file and annotate every flag.
- Supporting cast to explore: objdump (disassemble), size (section sizes), nm (symbol table), readelf (ELF structure).

Block 2 (10:30–12:00) — The Linker Script and Startup File

This is the deepest material of the day. It separates people who understand embedded from people who copy projects.

The problem the linker solves

Your .o files contain code and data but no addresses. The linker script — STM32F446RETX_FLASH.ld in your project — decides: code goes in flash at 0x08000000, variables go in SRAM at 0x20000000.

Three things to understand in the .ld file

- **The MEMORY block** — declares two regions: FLASH (512 KB at 0x08000000) and RAM (128 KB at 0x20000000). These match the memory map chapter of RM0390 exactly.
- **The SECTIONS block** — places .isr_vector first in flash (the vector table must be first; the Cortex-M4 reads its initial stack pointer and reset address from offsets 0 and 4 at power-up). Then .text (code), .rodata (constants), .data (initialized globals), .bss (zero-init globals).
- **The LMA/VMA split on .data** — .data has two addresses. Its load address (LMA) is in flash (initial values must survive power-off). Its run address (VMA) is in RAM (variables must be writable). The line >RAM AT> FLASH expresses this. This is the key insight.

The startup file

Who copies .data from flash to RAM? Not hardware — software. Open startup_stm32f446retx.s and read Reset_Handler. Roughly 30 lines of assembly: set the stack pointer, loop copying .data from its flash home to its RAM home, loop zeroing .bss, call SystemInit, branch to main(). That is the entire story of how your program starts before main() — and CubeIDE never asked you to think about it.

Resource

Memfault Interrupt — 'Zero to main()' series, parts 1 and 2 (interrupt.memfault.com, search 'zero to main'). The best written treatment of this topic in existence.

Hands-on

Annotate your project's .ld file with comments explaining every section. Trace Reset_Handler line by line, matching each label (_sdata, _edata, _sidata, _sbss, _ebss) back to where the linker script defines it. Those symbols are the handshake between the two files.

Block 3 (13:00–14:00) — Binutils Autopsy on Your Own Blink

Take the .elf from your week-2 blink project and interrogate it from a terminal.

Commands to run

```
arm-none-eabi-size firmware.elf
arm-none-eabi-objdump -h firmware.elf
arm-none-eabi-objdump -d firmware.elf | less
arm-none-eabi-nm firmware.elf | sort
```

What to verify with your own eyes

- .isr_vector sits at 0x08000000. The first word is your initial stack pointer (top of SRAM, ~0x20020000). The second word is the address of Reset_Handler with the low bit set (Thumb mode indicator).

- Find main() in the disassembly. Read your GPIO register writes as actual LDR/STR ARM assembly instructions.
- In section headers, confirm .data's LMA and VMA differ — exactly as the linker script promised.
- Run objcopy -O binary yourself and check the .bin file size against what arm-none-eabi-size reported.

Checkpoint question

If you add a large initialized global array, which numbers in arm-none-eabi-size grow, and why does it consume both flash and RAM? Answer: data grows (RAM for the writable copy) and flash grows (initial values baked in via LMA). bss should NOT grow — that's for uninitialized data only, which costs no flash. If you can answer this crisply, the morning worked.

Block 4 (14:00–15:30) — Debug Plumbing: ST-Link, SWD, OpenOCD, GDB

The debug chain

GDB (on your PC) ↔ TCP:3333 ↔ GDB server (OpenOCD) ↔ USB ↔ ST-Link probe ↔ SWD (SWDIO + SWCLK) ↔ STM32 core. Each link speaks a different protocol; every tool is a translator between two of them.

Understanding the hardware

Your Nucleo is really two boards: the small section above the break-off line is a complete ST-Link v2-1 debug probe (its own STM32 running probe firmware), wired to the target F446 by two signals — SWDIO and SWCLK. SWD gives the probe godlike power: halt the core, read/write any address, set hardware breakpoints, program flash — all while your firmware has no idea it's being watched. This is why the debugger can inspect a hard-faulted chip.

GDB itself knows nothing about ST-Link or SWD. It speaks only the GDB remote serial protocol over TCP. A GDB server (OpenOCD, or ST's proprietary ST-LINK GDB server that CubeIDE defaults to) translates between GDB's 'read memory at X' requests and USB commands for the probe.

Hands-on — drive the entire chain without CubeIDE (45 min)

Install OpenOCD (apt install openocd / brew install openocd / xPack on Windows). Then:

```
# Terminal 1 — start the GDB server
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg

# Terminal 2 — connect GDB
arm-none-eabi-gdb firmware.elf
(gdb) target extended-remote :3333
(gdb) monitor reset halt
(gdb) load                        # flashes your chip — no IDE!
(gdb) break main
(gdb) continue
(gdb) info registers
(gdb) x/8wx 0x08000000            # read flash from start
```

```
(gdb) x/wx 0x40020014            # read GPIOA ODR live off silicon
```

That x/wx command reads GPIOA's ODR register live off the silicon — verify the address against RM0390 yourself. Step with next, watch the LED, examine $pc. Every button in CubeIDE's debug perspective is one of these commands.

Block 5 (15:30–17:00) — Makefile from Scratch, Then PlatformIO

Assemble the day's parts. Make a fresh directory with only: main.c, the startup .s file, the .ld linker script (all copied from your CubeIDE project), and a Makefile you write yourself.

Minimal working Makefile

```
CC      = arm-none-eabi-gcc
OBJCOPY = arm-none-eabi-objcopy
CFLAGS  = -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16 \
          -ffunction-sections -fdata-sections -Og -g3 -Wall
LDFLAGS = -T STM32F446RETX_FLASH.ld --specs=nosys.specs -Wl,--gc-sections

SRCS    = main.c startup_stm32f446retx.s
OBJS    = $(SRCS:.c=.o)

all: firmware.bin

firmware.elf: $(OBJS)
■$(CC) $(CFLAGS) $(LDFLAGS) -o $@ $^

firmware.bin: firmware.elf
■$(OBJCOPY) -O binary $< $@

flash: firmware.bin
■openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \
■  -c 'program firmware.bin 0x08000000 verify reset exit'

gdb: firmware.elf
■arm-none-eabi-gdb firmware.elf -ex 'target extended-remote :3333'

clean:
■rm -f *.o *.elf *.bin
```

When make flash blinks the LED, you have personally replaced STM32CubeIDE with about 25 lines of text.

Resource

bare-metal-programming-guide by Sergey Lyubka on GitHub (github.com/cpq/bare-metal-programming-guide) — a free, superb walkthrough doing exactly this exercise. The written companion this whole day deserves.

Mapping to PlatformIO / VS Code

Watch Phil's Lab's VS Code/PlatformIO setup video with new eyes. Notice that platformio.ini is just a declaration of the same five tools. PlatformIO downloads the same arm-none-eabi toolchain into ~/.platformio/packages, drives it with its own build scripts instead of make, and debugging in VS Code is the Cortex-Debug extension acting as a GUI over the same GDB/OpenOCD chain you drove by hand today. There is nothing left to learn for the transition — you already own every layer.

End-of-Day Checkpoint

You are done when you can answer these cold, out loud, without notes:

- What decides that code lives at 0x08000000 — hardware or software?
- Why does .data occupy both flash and RAM, and what code performs the copy?
- What is the difference between the .elf and the .bin? Which one does GDB want?
- When you click the bug icon in CubeIDE, name every process that starts and every protocol on every link down to the chip.
- What does the CPU do in the first two bus reads after reset?

GitHub artifact

Commit the Makefile blink as stm32-no-ide with a README answering exactly those questions. It complements your week-2 register-level blink repo perfectly, and 'I can build and debug an ARM target without an IDE' is a genuinely strong interview signal.

Your Follow-Up Question

> *"PS C:\Users\ethan\Documents\Firmware-60-day-Challenge\Week 2 - STM32 toolchain, ARM Cortex-M\2_gpio_output> arm-none-eabi-size 2_gpio_output.elf — 'arm-none-eabi-size' is not recognized as the name of a cmdlet, function, script file, or operable program."*

That's expected — the toolchain is installed, but only CubeIDE knows where it is. It lives buried in CubeIDE's plugins folder and was never added to your system PATH, so PowerShell can't find it. Two-minute fix:

Fix option 1 — add it to PATH permanently (recommended)

Find the toolchain folder. In PowerShell:

```
Get-ChildItem "C:\ST\STM32CubeIDE*" -Recurse -Filter "arm-none-eabi-gcc.exe" | Select-Object FullName
```

Copy the folder path (without the .exe). Then:

```
# Add to your PATH for this session only
$env:PATH += ";C:\ST\STM32CubeIDE_1.x.x\plugins\...\bin"

# Or permanently via System Properties > Environment Variables > Path
```

Fix option 2 — use CubeIDE's built-in terminal

In CubeIDE: Window → Show View → Terminal. This terminal inherits CubeIDE's PATH and already knows where the toolchain lives. arm-none-eabi-size will work immediately.

Verify it worked

```
arm-none-eabi-gcc --version
arm-none-eabi-size 2_gpio_output.elf
```

Quick Reference Card

Memory map (STM32F446)

- 0x08000000 — Flash (512 KB). Code, rodata, and the load image of .data all live here.
- 0x20000000 — SRAM (128 KB). Stack, .bss, and the run-time copy of .data live here.
- 0x40000000+ — Peripheral registers. GPIO, RCC, UART, etc. — all memory-mapped.
- 0xE0000000+ — ARM core peripherals. NVIC, SysTick, DWT, ITM.

arm-none-eabi-size output columns

- **text** — code + rodata + the flash-stored copy of .data. Everything that lives in flash.
- **data** — initialized global/static variables. Costs RAM at runtime AND flash for the initial values.
- **bss** — zero-initialized globals. Costs RAM only; no flash cost.
- **dec / hex** — total of the above three.

Essential GDB commands

- target extended-remote :3333 — connect to OpenOCD
- monitor reset halt — halt the chip via OpenOCD
- load — flash the .elf
- break main / continue / next / step — standard flow control
- info registers — dump all CPU registers
- x/8wx 0x08000000 — examine 8 words of memory as hex
- x/wx 0x40020014 — read GPIOA ODR register live
- info threads / bt — thread list / backtrace (useful in FreeRTOS later)

Toolchain command cheat sheet

```
arm-none-eabi-size firmware.elf          # section sizes
arm-none-eabi-objdump -h firmware.elf    # section headers with addresses
arm-none-eabi-objdump -d firmware.elf    # disassemble
arm-none-eabi-nm firmware.elf | sort     # symbol table
arm-none-eabi-readelf -a firmware.elf    # full ELF dump
arm-none-eabi-objcopy -O binary f.elf f.bin  # strip to raw binary
```